

Centro Paula Souza – Fatec Bebedouro

Tecnologia em Big Data no Agronegócio

Algoritmos Avançados – Lista de Exercícios: Listas, Tuplas e Dicionários

Prof. Me. Douglas Samuel Gonçalves

2º Semestre / 2026

1. Classificação e Filtragem de Sensores (Listas e Tuplas)

Uma estação meteorológica agrícola recebe uma lista de tuplas contendo o identificador do sensor e a umidade do solo registrada:

```
sensores = [("S1", 45.2), ("S2", 28.0), ("S3", 60.5), ("S4", 19.8), ("S5", 33.1)]
```

Escreva um programa em Python que percorra essa lista e separe os registros em duas novas listas:

- a) **criticos**: sensores com umidade estritamente inferior a 30%;
- b) **adequados**: sensores com umidade igual ou superior a 30%.

Ao final, imprima a quantidade de sensores em cada categoria e a lista de sensores críticos.

2. Estatísticas de Produtividade Agrícola (Listas)

Dada uma lista com os valores de produtividade (em sacas por hectare) de 10 talhões de uma fazenda de soja:

```
produtividades = [68.5, 72.0, 55.4, 81.2, 74.0, 63.8, 79.5, 58.0, 84.1, 70.5]
```

Escreva um programa em Python que, sem utilizar funções nativas como `max()`, `min()` ou `sum()`, determine:

- a) A maior e a menor produtividade registrada;
- b) A produtividade média geral da propriedade;
- c) A quantidade de talhões que obtiveram produtividade acima da média calculada.

3. Cálculo de Distância Geoespacial (Tuplas)

Pontos de amostragem no campo são representados por tuplas contendo coordenadas (x, y) em quilômetros. Escreva uma função que receba duas tuplas $P_1 = (x_1, y_1)$ e $P_2 = (x_2, y_2)$ e retorne a distância euclidiana entre eles:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Em seguida, dada a lista de coordenadas de 5 armadilhas:

```
armadilhas = [("A1", (2, 3)), ("A2", (5, 8)), ("A3", (1, 1)), ("A4", (7, 4)), ("A5", (3, 6))]
```

Identifique qual armadilha está mais distante da sede da fazenda, localizada em (0,0).

4. Contagem de Frequência de Ocorrências (Dicionários)

Um técnico agrícola anotou em uma lista as pragas identificadas em inspeções:

```
inspecoes = [  
    "Lagarta", "Mosca-branca", "Lagarta",  
    "Percevejo", "Bicudo", "Lagarta", "Percevejo"  
]
```

Escreva um programa em Python que construa um dicionário onde cada chave seja o nome da praga e o valor seja o total de detecções. Ao final, imprima a praga mais frequente.

5. Histórico de Culturas por Talhão (Dicionários e Listas)

Dada uma lista de tuplas registrando o histórico no formato (talhao, cultura, ano):

```
historico = [  
    ("T-01", "Soja", 2024), ("T-02", "Milho", 2024),  
    ("T-01", "Milho", 2025), ("T-03", "Cana", 2025),  
    ("T-01", "Algodao", 2026)  
]
```

Construa um dicionário onde cada chave seja o código do talhão e o valor associado seja uma lista com as culturas plantadas nele (sem duplicar nomes de culturas no mesmo talhão).

6. Transformação e Filtragem com Dictionary Comprehension

Uma cooperativa armazena a área declarada por produtor em um dicionário:

```
fazendas = {  
    "Fazenda A": 120.0, "Fazenda B": 45.0,  
    "Fazenda C": 85.5, "Fazenda D": 30.0, "Fazenda E": 210.0  
}
```

Utilizando exclusivamente **Dictionary Comprehension**:

- Crie um novo dicionário convertendo os valores de hectares para alqueires paulistas (1 alqueire = 2,42 ha);
- Crie um dicionário filtrado contendo apenas as propriedades com área > 50,0 ha.

7. Grade Geoespacial com Chaves Compostas (Tuplas e Dicionários)

A acidez do solo em uma grade regular é armazenada em um dicionário onde as chaves são coordenadas (*linha, coluna*) e os valores correspondem ao pH medido:

```
grade_solo = {  
    (0, 0): 6.2, (0, 1): 5.8, (0, 2): 6.0,  
    (1, 0): 6.5, (1, 1): 5.5, (1, 2): 6.1,  
    (2, 0): 6.3, (2, 1): 5.9, (2, 2): 6.4  
}
```

Escreva uma função que receba a grade e uma coordenada (*lin,col*) e retorne a média aritmética do pH entre a coordenada e seus 4 vizinhos imediatos (cima, baixo, esquerda, direita), ignorando posições fora dos limites da grade.

8. Inversão de Estrutura de Mapeamento (Dicionários)

Um sistema de telemetria mapeia os defensivos agrícolas e os talhões atendidos:

```
aplicacoes = {
    "Herbicida_X": ["T-01", "T-02"],
    "Fungicida_Y": ["T-02", "T-03", "T-04"],
    "Inseticida_Z": ["T-01", "T-04"]
}
```

Escreva uma função `inverter_mapeamento(d)` que receba esse dicionário e retorne um novo dicionário mapeando cada **talhão** para a lista de **defensivos** aplicados nele.

9. Modelagem de Registros com NamedTuple

Utilizando `namedtuple` do módulo `collections`, defina a estrutura para amostras de solo contendo os campos: `id`, `talhao`, `ph`, `mat_org` e `umidade`.

- Instancie uma lista com 4 registros de amostras coletadas em campo;
- Crie uma função que receba essa lista e retorne os identificadores (`id`) das amostras que necessitam de calagem ($\text{pH} < 6,0$) e possuem baixa matéria orgânica ($\text{mat_org} < 2,5\%$).

10. Representação de Grafo Ponderado via Dicionário de Adjacência

Uma usina modelou a malha de transporte entre frentes de colheita e a sede por meio de um grafo ponderado (lista de adjacência com dicionários aninhados):

```
grafo_logistica = {
    "Sede": {"Frente_1": 12, "Frente_2": 8},
    "Frente_1": {"Sede": 12, "Frente_3": 15},
    "Frente_2": {"Sede": 8, "Frente_3": 10},
    "Frente_3": {"Frente_1": 15, "Frente_2": 10}
}
```

Onde as chaves são os vértices e os valores são dicionários com vizinhos e distâncias em quilômetros. Escreva duas funções:

- Função `custo_rota(grafo, rota)`: recebe o grafo e uma lista com a sequência de vértices visitados (ex: `["Sede", "Frente_2", "Frente_3"]`) e calcula a distância total percorrida (retorne `-1` se alguma conexão não existir);
- Função `listar_conexoes(grafo, vertice)`: retorna uma lista de tuplas (`destino, distancia`) com os vizinhos do vértice informado, ordenados da menor para a maior distância.